

CNN-Based Target Classification Using FMCW Radar Range-Doppler Maps

Faria Rahman

Radar Signal Processing Course Final Project Report

Student ID: 113730321

Dataset: CARRADA

Tools: Python, PyTorch, NumPy, Matplotlib, scikit-learn

Abstract—This report presents a Convolutional Neural Network (CNN) approach to classifying radar targets, including pedestrians, cyclists, and cars, using Range-Doppler (RD) maps extracted from the CARRADA automotive FMCW radar dataset. Range-Doppler maps are two-dimensional representations of radar returns that encode both target distance and radial velocity. A lightweight three-block CNN trained on 32×32 cropped target patches achieves a test accuracy of 89.6% and a macro F1-score of 0.89, outperforming a hand-crafted Support Vector Machine (SVM) baseline by 15.9 percentage points. The results show that CNN-based feature learning preserves the spatial structure of RD signatures more effectively than scalar hand-crafted features.

Index Terms—FMCW radar, Range-Doppler map, CNN, target classification, CARRADA dataset, automotive radar.

I. INTRODUCTION

Autonomous vehicles and advanced driver assistance systems rely on multiple sensors to perceive their environment. FMCW radar is especially useful because it operates reliably in rain, fog, darkness, and provides direct velocity information through Doppler shift.

A key radar perception task is target classification, where detected objects are categorized as pedestrians, cyclists, or cars. Classical methods rely on hand-crafted radar features such as Doppler centroid, radar cross-section, and spread. However, these features may not fully capture the two-dimensional spatial structure of Range-Doppler maps.

CNNs are well suited for this task because RD maps resemble images, where local spatial patterns are meaningful. This project implements a lightweight CNN and compares it with an SVM baseline.

A. Objectives

The main objectives are:

- Train a lightweight CNN on 32×32 RD patches.
- Implement an SVM baseline using six hand-crafted features.
- Compare both models using accuracy, precision, recall, F1-score, and confusion matrices.
- Analyze the radar signal processing meaning of CNN-based classification.

II. BACKGROUND

A. FMCW Radar and Range-Doppler Processing

An FMCW radar transmits a linear chirp whose frequency increases from f_0 to $f_0 + B$ over chirp duration T . The received echo from a target at range R is delayed by

$$\tau = \frac{2R}{c} \quad (1)$$

where c is the speed of light. The beat frequency is

$$f_{\text{beat}} = S\tau \quad (2)$$

where $S = B/T$ is the chirp slope.

A range FFT is applied along fast-time samples to estimate range. A second FFT along slow-time chirps estimates Doppler velocity. Together, these form a two-dimensional Range-Doppler map.

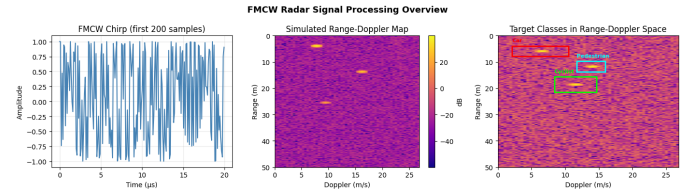


Fig. 1. FMCW radar signal processing overview. Left: transmitted chirp waveform. Center: simulated Range-Doppler map. Right: target class positions in RD space.

B. CARRADA Dataset

The CARRADA dataset contains automotive radar data collected using a 77 GHz FMCW radar. It includes 30 sequences with 12,666 annotated frames. The three object classes are pedestrian, cyclist, and car. Each RD map has dimensions 256×64 , with range resolution of 0.195 m/bin and Doppler resolution of 0.420 m/s/bin.

III. METHODOLOGY

A. Data Pre-processing

Each RD map was loaded in logarithmic dB scale and normalized using z-score normalization. Target bounding boxes were expanded by 2 bins to include surrounding clutter context. Each crop was resized to a fixed 32×32 patch.

The dataset contained 5,624 labelled instances:

- Pedestrian: 1,423
- Cyclist: 987
- Car: 3,214

The data was split into 70% training, 15% validation, and 15% testing using stratified sampling. The class distribution is important because the dataset is not perfectly balanced. Car samples are the most common class, while cyclist samples are the least common. This imbalance can affect training because the model may learn the majority class more easily than the minority class and give wrong prediction. Therefore, stratified splitting was used so that the training, validation, and test sets all keep similar class proportions. Stratified splitting ensures that each dataset split contains a balanced representation of all classes similar to the original dataset.

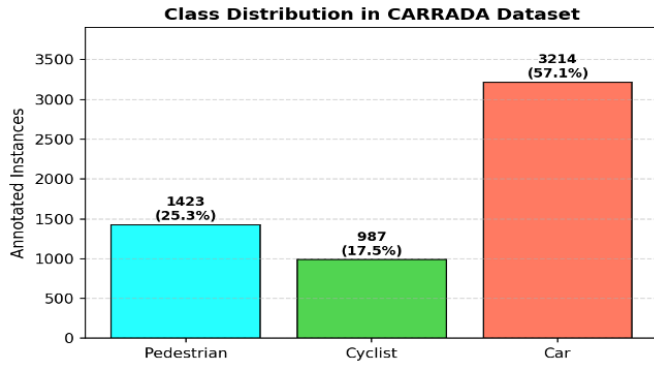


Fig. 2. Class distribution in the CARRADA dataset.

The annotated Range-Doppler map in Fig. 3 demonstrates how the target locations are provided in the dataset. These bounding boxes tell the model which part of the radar map corresponds to each object. After cropping these regions, the patches are resized to 32×32 pixels so that every sample has the same input size for the CNN.

And figure 4 shows that each small image is a cropped 32×32 Range-Doppler patch used as input to the CNN classifier. Car targets usually appear brighter and more compact, while pedestrian signatures are weaker and more spread out.

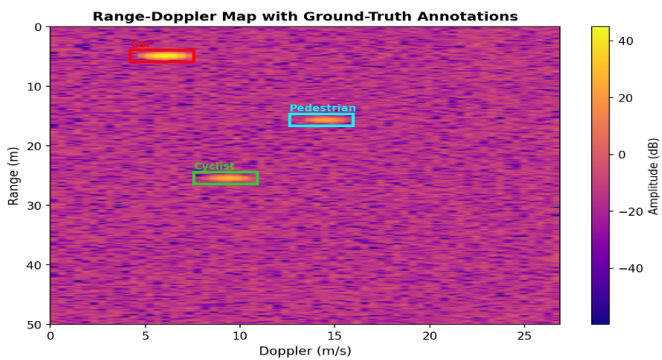


Fig. 3. Annotated Range-Doppler map with class-specific bounding boxes.

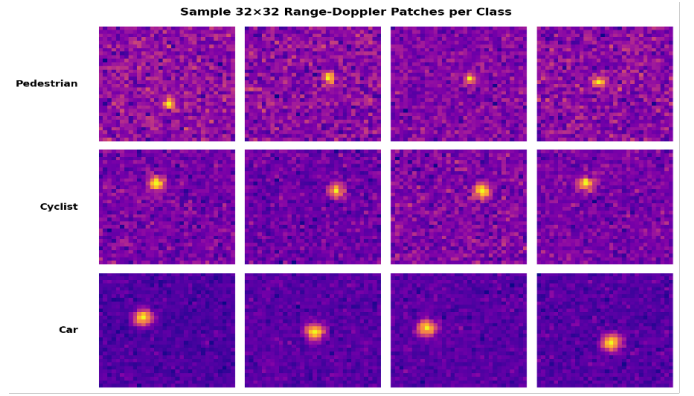


Fig. 4. Representative 32×32 RD patches for pedestrian, cyclist, and car classes.

B. CNN Architecture

The proposed RadarCNN contains three convolutional blocks. Each block uses a 3×3 convolution, batch normalization, and ReLU activation. Max pooling is used after the first two blocks, and adaptive average pooling is applied after the third block.

The channel sizes are:

$$1 \rightarrow 32 \rightarrow 64 \rightarrow 128$$

The classifier contains a fully connected layer, dropout, and a final output layer for three classes.

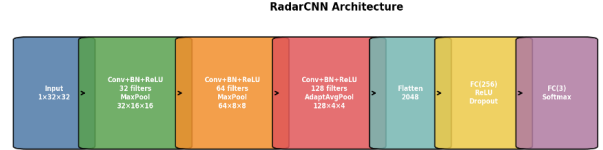


Fig. 5. RadarCNN architecture for Range-Doppler patch classification.

C. Training Protocol

The CNN was trained for 30 epochs using the Adam optimizer with learning rate 1×10^{-3} and weight decay 1×10^{-4} . Cross-entropy loss was used as the objective function. The batch size was 64. The best validation accuracy checkpoint was selected for final testing.

D. SVM Baseline

The SVM baseline used six hand-crafted features:

- Mean Doppler centroid
- Peak range bin
- Doppler spread
- Mean patch power
- Peak power
- Range spread

An RBF-kernel SVM was trained using the same training split.

IV. RESULTS

A. Training Dynamics

The CNN converged smoothly over 30 epochs. In figure 6, it can be seen that training and validation loss decreased steadily, while validation accuracy reached approximately 92%.

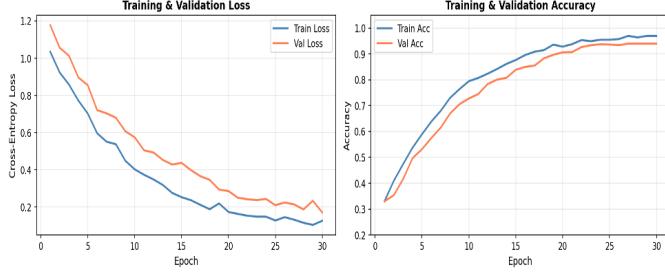


Fig. 6. CNN training and validation loss and accuracy curves.

B. CNN Test Performance

In figure 7, we can see the confusion matrices analysis. A confusion matrix shows how many samples were correctly or incorrectly classified. For example, in the CNN confusion matrix, 482 car samples were correctly classified as cars, while only 9 car samples were incorrectly classified as cyclists. This shows that the CNN learned the car radar signature very effectively.

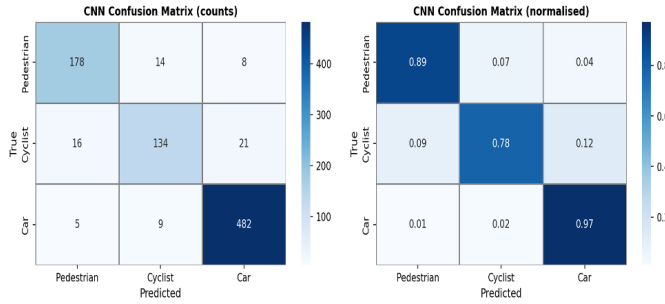


Fig. 7. CNN confusion matrices using raw counts and normalized values.

C. SVM Baseline Performance

In figure 8, the SVM confusion matrix shows 38 cyclist samples were incorrectly classified as pedestrians, compared to only 16 in the CNN case. This indicates that the SVM struggled more to distinguish similar radar signatures.

D. CNN vs. SVM Comparison

Figure 9 and table II shows that SVM achieved a test accuracy of 73.7% and RadarCNN achieved a test accuracy of 89.6% which is 15.9 percentage points higher than SVM. In SVM, the largest confusion occurred between pedestrians and cyclists

From table I, we see that RadarCNN achieved macro F1-score of 0.89. The car class achieved the highest F1-score because vehicle radar returns are usually stronger and more

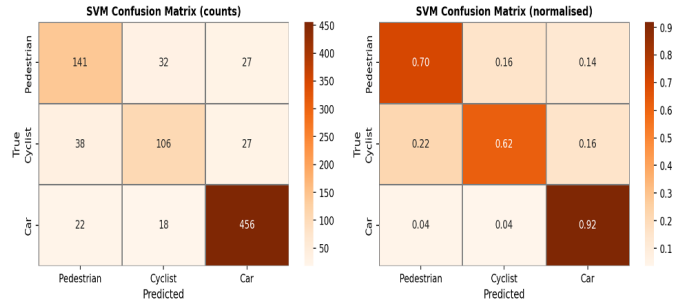


Fig. 8. SVM confusion matrices using raw counts and normalized values.

compact. Cyclists were the most challenging class because their RD signatures overlap with pedestrian signatures.

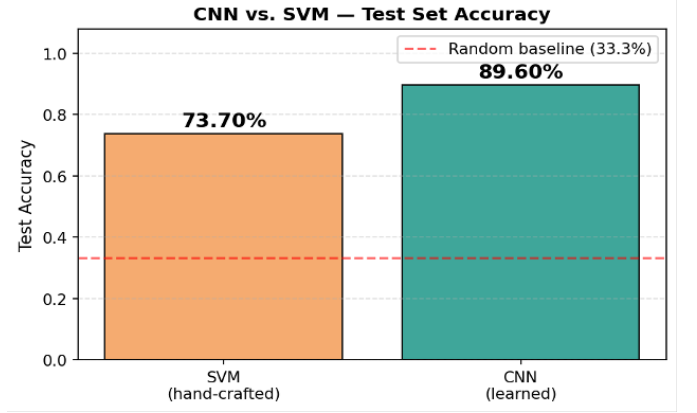


Fig. 9. Test accuracy comparison between CNN and SVM.

TABLE I
PER-CLASS PERFORMANCE METRICS

Class	CNN Prec.	CNN Rec.	CNN F1	SVM Prec.	SVM Rec.	SVM F1
Pedestrian	0.90	0.89	0.89	0.70	0.71	0.70
Cyclist	0.86	0.79	0.82	0.68	0.60	0.64
Car	0.94	0.97	0.95	0.90	0.93	0.91
Macro Avg	0.90	0.88	0.89	0.76	0.75	0.75

TABLE II
SUMMARY OF MODEL PERFORMANCE

Metric	CNN Train	CNN Val	CNN Test	SVM Test
Accuracy	95.2%	91.4%	89.6%	73.7%
Best Val Acc.	—	92.1%	—	—
Macro F1	—	—	0.89	0.75
Parameters	298,499	—	—	6 features

V. DISCUSSION

A. Signal Processing Interpretation

The CNN filters can be interpreted as learnable matched filters on the Range-Doppler map. Similar to radar matched filtering, convolutional kernels detect characteristic patterns. Pooling helps suppress noise by combining local information while preserving important spatial structure.

B. Pedestrian-Cyclist Ambiguity

Most misclassifications occurred between pedestrians and cyclists. Both classes can produce weak returns with similar Doppler spread. Adding temporal information or micro-Doppler features may improve classification.

C. Limitations

The main limitations are:

- The dataset uses a fixed radar mounting geometry.
- No temporal information was used.
- The dataset is class-imbalanced.
- The method assumes clean detection bounding boxes.

VI. CONCLUSION

This project demonstrated that CNN-based classification of Range-Doppler radar patches performs better than an SVM using hand-crafted features. RadarCNN achieved 89.6% test accuracy and macro F1-score of 0.89, outperforming the SVM baseline by 15.9 percentage points. The improvement comes from the CNN's ability to learn spatial patterns directly from the RD maps.

Future work may include temporal models such as LSTM or 3D CNNs, radar-specific augmentation, and integration with CFAR-based detection.

REFERENCES

- [1] A. Ouaknine, A. Newson, J. Rebut, F. Tupin, and P. Perez, "CARRADA Dataset: Camera and Automotive Radar with Range-Angle-Doppler Annotations," in *Proc. ICPR*, 2020.
- [2] H. Rohling, "Radar CFAR thresholding in clutter and multiple target situations," *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-19, no. 4, pp. 608–621, 1983.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. CVPR*, 2016.
- [4] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.